

HookUX Full Stack Developer - What to Learn

MUST Learn (Class A keywords with no direct project experience)

- **Vue.js** – a component-based frontend framework, similar in spirit to React (both use a virtual DOM, reactive state, and a component tree). Vue's Options/Composition API maps closely to React Hooks – `ref/reactive` $\approx$ `useState`, `watch` $\approx$ `useEffect`. Build one small CRUD app with Vue 3 + Composition API to get comfortable with templates (`v-if`, `v-for`, `v-model`) and single-file components (`.vue`).
- **AngularJS (1.x)** – the legacy (pre-2016) Angular, NOT modern Angular. Uses two-way data binding, `$scope`, controllers, and directives – a very different mental model from React's one-way data flow. Since it's legacy tech, the 20% to know: modules (`angular.module`), controllers, `ng-repeat/ng-model`, and services/factories for shared logic. Skim the official AngularJS docs; don't over-invest since it's a maintenance-mode framework.
- **Django** – Python's batteries-included web framework (models, views, templates, admin panel, ORM). If you know Express.js, map it directly: Django URLconf $\approx$ Express routes, Django views $\approx$ Express route handlers, Django ORM $\approx$ Prisma/Mongoose. Build one small REST API with Django REST Framework (DRF) to see the parallels to your Express/REST work.
- **WebGL** – low-level browser API for GPU-accelerated 2D/3D graphics, relevant here for a gamified investing simulation UI. You already have canvas/graphics experience from Pixxel (Fabric.js). The bridge: Fabric.js sits on top of Canvas 2D; WebGL is lower-level and 3D-capable. Start with Three.js (a WebGL wrapper, much friendlier than raw WebGL) to build one animated scene.
- **MySQL** – you know PostgreSQL (Prisma + raw SQL in Skimly) and relational modeling already; MySQL's SQL dialect is 95% the same. The 20%: MySQL-specific syntax quirks (`AUTO_INCREMENT` vs `SERIAL`, `LIMIT` syntax, storage engines like InnoDB). Not a real learning gap – mention this parallel confidently in interviews.

GOOD to Learn (strengthens the application, not blocking)

- **Azure & Google Cloud** – you've deployed on AWS (Secrets Manager in Echo) and Vercel. Skim Azure App Service and GCP Cloud Run – both map to "deploy a container/app," so the concepts transfer; you mainly need enough vocabulary to discuss trade-offs (managed compute, storage, IAM basics).
- **Blockchain/crypto fundamentals** – relevant since HookUX simulates crypto investing. You don't need to write smart contracts – just understand wallets, price feeds/APIs (e.g. CoinGecko API), and how a "simulated" trading engine differs from a real exchange integration (no real funds, mocked order matching).
- **Kubernetes** – you have real Docker experience (Vibe's E2B sandbox template). K8s is the next layer up (orchestrating many containers). Learn the core nouns – Pod, Deployment, Service – via a single `kubect1` tutorial; deep expertise isn't expected at 0-3 YOE.

Fast-Ramp Note

Your core stack (React, Node/Express, REST APIs, PostgreSQL/MongoDB, Git, Agile) already covers the majority of this JD's day-to-day work. Vue and AngularJS are additional framework surface area, not new paradigms – your component-thinking from React transfers directly. Prioritize Vue.js and Django since those are the most likely to come up in a technical screen for this role.